

Guía del lenguaje .bldh

BuLletDasHer · Sistema de creación de niveles del Practice Mode de BulletDasher

Esta guía explica de punta a punta el formato **.bldh**: la sintaxis de cada bloque, los parámetros de cada enemigo/peligro, el sistema de coordenadas, y cómo se arma y edita una lista desde el Practice Mode del juego.

1. ¿Qué es un archivo .bldh?

Un archivo **.bldh** es una lista de instrucciones que describe una secuencia de eventos del juego: qué enemigos invocar, cuándo, con qué parámetros exactos, cuánto esperar entre uno y otro, y (opcionalmente) lógica de control como repeticiones y condicionales. Se arma desde el **Creador de Niveles** dentro del Practice Mode, y se puede exportar/cargar como archivo de texto plano con extensión `.bldh`.

Sirve para diseñar **desafíos específicos**: un patrón de esquiva concreto, una secuencia coreografiada de peligros, una prueba de resistencia armada a mano — y después compartir ese desafío como un simple archivo de texto.

Estructura general

```
INICIO&bloque1&bloque2&bloque3&...&FIN
```

El separador entre bloques es el carácter **&** (y no un guion), porque los parámetros de posición de los bloques pueden ser números negativos y un guion generaría ambigüedad al leer el archivo.

2. Sistema de coordenadas

Todos los parámetros de posición (x , y) se escriben con el **centro del canvas como origen (0,0)** — como un plano cartesiano normal, no como la esquina superior izquierda que usa internamente el canvas del navegador.

Eje	Valor positivo	Valor negativo
X	Hacia la derecha	Hacia la izquierda
Y	Hacia arriba	Hacia abajo

El editor convierte estos valores a las coordenadas internas del canvas automáticamente — no hay que hacer ninguna cuenta manual al escribir los parámetros.

3. Bloques básicos

3.1 · invocar()

Invoca un enemigo o peligro. Si se dejan los parámetros vacíos, el spawn es **aleatorio**, exactamente igual que el botón clásico de invocar del Practice Mode. Si se completan los parámetros, el enemigo aparece exactamente donde se pida.

<code>invocar(purp())</code>	→ spawn aleatorio (comportamiento clásico)
<code>invocar(purp(120, -80))</code>	→ purp exacto en x=120, y=-80
<code>invocar(rock(0,0,3))</code>	→ asteroide tier 3 en el centro

3.2 · esperar()segundos

Pausa el avance de la lista durante N segundos antes de seguir con el siguiente bloque. El juego y el jugador NO se congelan — solo se pausa el avance de la propia lista.

`esperar(3)segundos`

4. Parámetros por enemigo

Cada tipo de enemigo/peligro acepta su propia lista de parámetros, en este orden, separados por comas dentro del paréntesis:

ID	Parámetros	Notas
purp	x, y	Enemigo lento clásico.
redst	x, y	Proyectil rápido que apunta al jugador.
laser	x, y, angulo, aparicion, duracion	Ángulo en grados convencionales (0° = derecha). "aparicion" y "duracion" en segundos: tiempo de carga antes de activarse y tiempo que dura activo.
lamov	lado, vel	lado: 0 = izquierda, 1 = derecha. Láser que se desplaza por el canvas.
extcn	x, y, tiempo	Evento de extinción/sombra; tiempo en segundos, x,y ubican la zona segura.
banex	x, dir	dir: 0 = izquierda, 1 = derecha. Banda de extinción horizontal.
pepes	x, y, tiempo	Bouncer (rebotador); tiempo de vida en segundos.
rock	x, y, tier	Asteroide; tier de 1 a 3 (tamaño/resistencia).
brskr	x, y, vidas	Berserker; "vidas" son sus puntos de vida (hp).

En cualquier enemigo, dejar TODOS los parámetros vacíos — `invocar(purp())` — hace que el spawn sea aleatorio, como el modo clásico. Esto no cambia ningún comportamiento existente: los parámetros son un plus para armar desafíos, no un reemplazo del modo normal.

5. Bloque Repetir

Repite un tramo de bloques ya puestos en la lista, sin tener que volver a escribirlos.

Repetir (veces) veces (dir) bloques (cantidad)	
Parámetro	Significado
veces	Cuántas veces se repite el tramo en total.
dir	1 = apunta HACIA ARRIBA (repite los "cantidad" bloques anteriores a este). 2 = apunta HACIA ABAJO (repite los "cantidad" bloques siguientes a este).
cantidad	Cuántos bloques abarca el tramo a repetir (cuenta cualquier tipo de bloque, incluyendo otros Repetir/Complejo — así se pueden anidar loops).

Ejemplo — repetir 3 veces los 2 bloques anteriores:

```
INICIO&
invocar(purp(100,0))&
esperar(1)segundos&
Repetir(3)veces(1)bloques(2)&
FIN
```

Este ejemplo invoca un purp y espera 1 segundo, y ese par de acciones se repite 3 veces en total (la pasada natural + 2 repeticiones más).

6. Bloque Complejo

Todo bloque Complejo empieza con el prefijo **Complejo.** seguido de una de las siguientes variantes:

6.1 · Confeti

Dispara un efecto de confeti en pantalla.

```
Complejo.Confeti
```

6.2 · Texto("...")por(segundos)

Muestra un texto en el canvas durante la cantidad de segundos indicada.

```
Complejo.Texto("¡Cuidado!")por(2)
```

6.3 · MusicHTMLFunc(archivo.mp3)

Cambia la música de fondo al archivo indicado.

```
Complejo.MusicHTMLFunc(boss_theme.mp3)
```

6.4 · Si(condición)Entonces(N)Sino(M)

Condición: evalúa una condición y ejecuta un tramo u otro según el resultado. El Entonces (N) es obligatorio (toma los N bloques siguientes al Si). El Sino (M) es opcional — se puede dejar vacío o en 0 si no hay rama alternativa.

Condición	Verdadera cuando...
vacio	Hay al menos un enemigo vivo en pantalla (cuenta purps, redsts, pepes, berserkers y asteroides; NO cuenta lasers/extcn/banex, que son hazards de entorno).
vidas>#	Las vidas (hp) actuales del jugador son mayores a #.
vidas=#	Las vidas (hp) actuales del jugador son iguales a #.
vidas<#	Las vidas (hp) actuales del jugador son menores a #.

Ejemplo — si no quedan enemigos, festejar; si no, esperar un poco más:

```
INICIO&
Complejo.Si(vacio)Entonces(1)Sino(1)&
Complejo.Confeti&
esperar(2)segundos&
FIN
```

6.5 · Repetirinfvec(es)(dir)bloques(N)

Repetición infinita — misma lógica de dirección que el bloque Repetir (1 = arriba, 2 = abajo), pero sin límite de vueltas. Útil para tramos que deben correr todo el resto de la partida.

```
Complejo.Repetirinfvec(es)(2)bloques(3)
```

Cuando la lista termina de correr (sin bloques infinitos), el juego sigue con normalidad: spawns aleatorios de siempre, como si el .bldh nunca hubiera estado activo.

7. El editor en el Practice Mode

7.1 · Activar el creador de niveles

Debajo del botón grande de invocar hay un switch sin texto. Al activarlo, el botón de invocar deja de spawnear directo y en cambio agrega un bloque invocar (. . .) a la lista que se está armando.

7.2 · Caja del creador

Botón	Función
■ ■	Borra toda la lista.
■	Agrega un bloque de espera de 1 segundo, o le suma 1 segundo más al último si ya hay uno puesto justo ahí.
■	Borra o decrementa el bloque de arriba de la vara. Si es un bloque con parámetros ya definidos, la lista brilla en rojo y no se borra hasta vaciar esos parámetros.
■	Abre un selector de archivos para cargar un .bldh existente.
■	Descarga la lista actual como archivo .bldh.
■	Agrega un bloque Repetir.
■	Agrega un bloque Complejo.

Tocar cualquier bloque de la lista (invocar, Repetir o Complejo) abre un campo de texto para escribir/editar sus parámetros a mano.

7.3 · La vara de inserción

En la lista aparece una barra brillante entre los bloques: marca dónde va a caer el **próximo** bloque que se agregue. Tiene dos flechitas propias (▲ / ▼) para moverla un lugar hacia arriba o hacia abajo — puede llegar hasta arriba de todo o quedar debajo del último bloque. Por defecto arranca abajo del todo, así que si nunca se toca, el editor se comporta como una lista normal que crece hacia abajo.

Moverla a la mitad de la lista permite insertar o reordenar bloques en medio de un .bldh ya armado, sin tener que borrar y rehacer todo lo de después.

7.4 · Correr la lista

Con el creador activo aparece un panel a la derecha con el botón ■, que corre la lista armada desde el principio. Debajo tiene un segundo switch, verde: si está activo, cada vez que la partida se reinicia (típicamente al revivir tras un game over) el .bldh cargado se corre automáticamente solo, sin tener que tocar ■ de nuevo — pensado para practicar un desafío específico en loop, una y otra vez.

8. Ejemplo completo

Un desafío corto: aparece un texto de aviso, se invocan dos purps enfrentados, se espera, y luego se repite un patrón de proyectiles tres veces.

```
INICIO&
Complejo.Texto("Prepárate")por(2)&
esperar(2)segundos&
invocar(purp(-150,0))&
invocar(purp(150,0))&
invocar(redst(0,120))&
esperar(1)segundos&
Repetir(3)veces(1)bloques(2)&
Complejo.Confeti&
FIN
```

9. Nota técnica: cómo se ejecuta la lista

La lista no se "desenrolla" antes de arrancar — se ejecuta con un intérprete real: un puntero de programa que recorre la lista y puede saltar hacia atrás o adelante (como una CPU). Esto es lo que permite loops infinitos (Repetirinfveces) sin colgar la pestaña: nunca hace falta generar una lista infinita de antemano.

Los bloques de control (Repetir, Complejo.Si, Repetirinfveces) se resuelven "gratis", sin consumir tiempo de juego. Solo invocar, esperar y los efectos puntuales de Complejo (Confeti, Texto, MusicHTMLFunc) avanzan el reloj de la lista.

Cuando la lista termina de correr, el juego sigue con normalidad: spawns aleatorios de siempre, como si el .bldh nunca hubiera estado activo.